

How to mod Rain World with Partiality

<u>1. PREPARATION</u>	<u>2</u>
<u>2. PARTIALITYMOD CLASS</u>	<u>2</u>
<u>3. DNSPY AND RAIN WORLD</u>	<u>3</u>
<u>4. HOOKING</u>	<u>5</u>
<u>5. NON-STATIC VARIABLE</u>	<u>6</u>
<u>6. ENUMEXTENDER</u>	<u>8</u>
<u>7. CONFIG SCREEN</u>	<u>10</u>
<u>EXTRA 1: ACCESSING PRIVATE VALUE IN HOOKING</u>	<u>15</u>
<u>EXTRA 2: MAKING A PARTIALITY PATCH</u>	<u>16</u>
<u>EXTRA 3: MODIFYING SLUGCAT'S GRAPHIC</u>	<u>18</u>
<u>EXTRA 4: ADDING CREATURES USING ENUMEXT</u>	<u>21</u>
<u>EXTRA 5: SUPPORTING TRANSLATION USING CONFIG MACHINE</u>	<u>27</u>
<u>EXTRA 6: PREPARING FOR VS CODE</u>	<u>29</u>

(Click Chapter to Jump)

1. Preparation

First, you'd need Visual Studio Community from Microsoft([Link](#)). If you are using Mac, you can use Visual Studio Code, although it will be lack of some features. Guide for using VS Code for mods is at [<Extra 6>](#) at the end of this tutorial. You can also use Notepad++ or any other program for scripting, but this tutorial won't/can't cover all that, and at that level, you would know how to compile a dll library with less beginner-friendly programs.

If you have a zero experience with programming, look up tutorials on YouTube or Google(e.g. [this](#)). It will help you get the hang of it. This tutorial assumes you know C# enough to understand basic scripts, the difference between `public` and `private`, and of course how to install Partiality Launcher.

After you install Partiality Launcher successfully, download `PublicityStunt` from RainDB's Tools tab([RainDB](#)). Disable everything else and enable only `PublicityStunt.dll` in Launcher and Apply Mods. This allows you to access non-public objects in rain world codes. Now, go into `<Rain World root folder>-[RainWorld_Data]-[Managed] Folder.` and copy `[HOOKS-Assembly-CSharp.dll]`(Required for Hooking), `[Partiality.dll]`(For using Partiality related classes), `[Assembly-CSharp.dll]`(To use classes in vanilla Rain World) and `[UnityEngine.dll]`(To use Unity related classes) to somewhere safe.

Open Visual Studio Community. Create a New Project, which is a `Class Library(.Net Framework)`. The framework must be `[.NET Framework 3.5]`. After you created your project with a witty name, find `[Project]-[Add Reference]` from the top. Add all five files for Reference. Now you are ready to create Rain World Mod.

2. PartialityMod class

First, you need to create `PartialityMod` class in your solution. These are example code. Create a cs file or rename `Class1.cs` which your IDE has automatically created and write like these:

```

using System.Collections.Generic;
using System.Linq;
using System.Text;
using Partiality.Modloader;
using UnityEngine;

public class MyMod : PartialityMod
{
    public MyMod()
    {
        instance = this;
        this.ModID = "MyMod";
        this.Version = "0100";
        this.author = "some dude";
    }

    public static MyMod instance; // Not necessary, but useful if you are planning to support Config Machine

    public override void OnEnable()
    {
        base.OnEnable();
        // Hooking codes would go here
    }
}

```

If you're using VS Community, click [Build]-[Build MyMod] on top menu to build your mod. It will create your mod dll in <Project Folder>-[bin]-[Debug] by default. Copy this to Rain World's Mods folder and apply your mod in Partiality Launcher. If you don't want to copy this every time you make a change, click [Project]-[Properties] from top menu, and select [Build Events] tab. Then paste this line below in [Post-build event command line]. Edit <Rain World Root Folder> to your setup, and separate folder names with \.

```
copy /Y "$(TargetPath)" "<Rain World Root Folder>\Mods\$(TargetName).dll"
```

Whenever you update your mod file, you can use Partiality Launcher Quick Launch.bat which clears metadata and reapplies mods then launches the game automatically. This quick launcher is intended for developing mods, so use it.

As Unity initializes, Partiality launches and runs OnEnable from your PartialityMod class. Now you can run anything in OnEnable. However, this alone does nothing yet. So let's mod Rain World.

3. dnSpy and Rain World

Download dnSpy([Link](#)), then extract it and run dnSpy (alternatively, you can use [ILSpy](#) or its Visual Studio [extension](#)). This will allow you to read DLL files, although variable names may be reinterpreted to less legible like 'num1', 'vector'.

Drag Assembly-CSharp.dll from your Reference folder (*not from* Managed folder – or having dnSpy turned on will prevent you patching the game) to Assembly Explorer on the left of the dnSpy window (if it is not there, go to [View]-[Assembly Explorer] or press [Ctrl+Alt+L] in dnSpy). What do you want to edit? It is up to your imagination, really, but you would have an idea for your mod. But beginners don't know where to look at in this vast library. So I will briefly explain how Rain World

works.

First of all, as Unity runs a `GameObject` with `MonoBehaviour` named `RainWorld` is initialized. This `MonoBehaviour` is a thin chain which connects Unity and Rain World the game which runs entirely on C# and Futile, which will be explained later. After it does some initialization process in `Start`, including checking `WorldChecksum` and loading shaders and resources, it initializes `ProcessManager`. Then every Unity frame `RainWorld` calls `ProcessManager.Update` with `Time.deltaTime` as a parameter. For the reference, `deltaTime` is how long Unity took to finish the last frame, and the unit is in second.

`ProcessManager` is *important*. Since `ProcessManager` is just initialized, it will put on `InputSelect` or `IntroRoll` depending on whether there is a preexisting option file or not. But in default, it updates `currentMainLoop`. It can be a menu or intro like I said, but mostly it is `RainWorldGame`, which ID is `ProcessManager.ProcessID.Game` for your future use. `ProcessManager` calls `RawUpdate` then.

Every class inheriting `MainLoopProcess` has `RawUpdate`, which calculates `timeStacker`, which indicates between last fixed frame (the exact time you divide a second with fps) and next fixed frame called `RawUpdate`. So this `timeStacker` stays between 0f to 1f. `RainWorldGame.RawUpdate` then calls `RainWorldGame.GrafUpdate` with `deltaTime`, then it calls `DrawUpdate` for every `RoomCamera` the game has, with `deltaTime` and `TimeSpeedFac`. `RoomCamera.DrawUpdate` then calls `VirtualMicrophone.DrawUpdate` for sound, and calls `Update` from every `SpriteLeaser` along with the position of the camera. Yes, in Rain World, every single sprite offsets to camera's position to simulate camera moving. And finally, `SpriteLeaser` calls `DrawSprites` from its owner, or in this case, `drawableObject`.

So this is how Graphics are updated from top-down:

```
Unity -> GameObject -> RainWorld -Update> ProcessManager -Update> RainWorldGame -RawUpdate> (every  
frame) -GrafUpdate> RoomCamera -DrawUpdate> SpriteLeaser -> IDrawable.DrawSprites
```

Let's roll back to `MainLoopProcess.RawUpdate`. When `timeStacker` exceeds 1f, which means a fixed frame has been passed, then it calls `RainWorldGame.Update`. From there every `AbstractRoom` and realized `Room` receive `Update`. Will explain what `Abstract` and `Realized` in Rain World mean in a bit. From `Room.Update` every `UpdatableAndDeletable` inside the room receive `Update`, then collision check happens between them.

```
Unity -> GameObject -> RainWorld -Update> ProcessManager -Update> RainWorldGame -RawUpdate>  
(once per every fixed frame; bool evenUpdate alternates each time) -Update> Room -Update>  
UpdatableAndDeletable.Update (and GraphicsModule.Update, PhysicalObject.GraphicsModuleUpdated)
```

While a whole region does get loaded in a single session, its every room cannot be simulated in full accuracy for performance's sake. So everything is happening in `Abstract`, and what you see, `Realized` and visualized graphics are just fancy representation to show you that

everything's looking pretty. In the code, slugcat's body just consists of two `BodyChunks`, and its tail is a decorative appendage that's reacting to the terrain so it doesn't go through the ground.

So you now learned how `RainWorldGame` works, then let's find what you want to edit then. Do you want to add new classes? Or edit existing ones?

4. Hooking

Hooking is the way you probably want to edit the code. Patching by `MonoMod` is a thing and it indeed is a quick and dirty way to modify codes, it has compatibility issues with other mods though. However this is still a method you can take and has one advantage, I will talk about Patching in [<Extra 2>](#). But let's go back to Hooking.

First, you need to know what method you want to hook. This depends on what kind of mod you want to create, but in this example, I will hook `Player.CanIPickThisUp`. This is how the method looks in `dnSpy`.

```
private bool CanIPickThisUp(PhysicalObject obj)
```

Now go to Visual Studio and write a static method with the same return type. Its name doesn't matter as long as you can remember what it does. Then for the first parameter `On.[class].orig_[method name]`, and the next parameter should be the class you are hooking unless the method you are hooking is `static`; in that case, skip this second parameter. Then third and farther on parameters should follow the original method you are hooking.

```
static bool CanIPickUpPatch(On.Player.orig_CanIPickThisUp orig, Player instance, PhysicalObject obj)  
// bool is the return type of original method in this example.
```

Now open the braces for your code. But first, you want to add a line to execute the original code of the game. This will also run other mods' codes if the user is using multiple mods at once, so it is a better practice to run the original code and workaround. To call the original method, use `Invoke` on the first parameter.

```
return orig.Invoke(instance, obj);  
// The first parameter is the class instance the method is being executed, and the other parameters are  
// original parameters. As you can see you can easily swap these parameters with other value to alter the  
// result of the original method.  
// In this example, Invoke returns a bool, which is the same value as the original method would. If the  
// return type is void, you can just run it once without 'return'. Or, you can save this value, write your  
// code, then use return later to execute your code after the original code.
```

So running codes before and after the original method looks like this.

```
public static bool CanIPickUpPatch(On.Player.orig_CanIPickThisUp orig, Player instance, PhysicalObject obj)
{
    // code that runs before original method
    bool result = orig.Invoke(instance, obj); // you can modify parameters before sending it to original
    // code that runs after original method
    return result; // you can return something else if you want
}
```

Let's do a fancier thing. Adding variable for slugcats. Since there will be four slugcats at max in the game, we can use a static array for this. Hook `Player.ctor` to initialize the variable, and access this variable in elsewhere with `playerState.playerNumber`.

```
public static int[] myInt;
static void CtorPatch(On.Player.orig_ctor orig, Player instance, AbstractCreature abstractCreature, World world)
{
    orig.Invoke(instance, abstractCreature, world);

    int slugcat = instance.playerState.playerNumber;
    myInt[slugcat] = 0;
}
```

Now let's actually hook this code. Go to `MyMod.OnEnable`, and write `On.[class].[method name] += new On.[class].hook_[method name]([hooking method]);` If a class you want to hook is inside a namespace, it'd be like `On.[namespace].[class].[method name]`.

```
public override void OnEnable()
{
    base.OnEnable();
    myInt = new int[4]; //In this example, I need to initialize static Array

    On.Player.ctor += new On.Player.hook_ctor(CtorPatch);
    On.Player.CanIPickThisUp += new On.Player.hook_CanIPickThisUp(CanIPickUpPatch);
}
```

For your information, with the help of `PublicityStunt`, you can use non-public stuff without Reflection. However, the players must install `PublicityStunt` too, to be able to use your mod. You can walk around this in two ways. Check [<Extra 1>](#) at the end of this tutorial to see more.

5. Non-static variable

When you want to edit `Creatures` or `PhysicalObjects`, however, `Array` with four indexes won't be sufficient. You will need to use a `Dictionary`. `Dictionary` works like `Array`, but instead of indexes it uses unique keys which can be any type. However, creating `Dictionary` for each variable is unproductive. So we use a `struct`. The following example is from the old version of `Lancer`.

```

public override void OnEnable()
{
    // Hooking codes...

    SpearBeamState = new Dictionary<Spear, BeamState>();
}
public static Dictionary<Spear, BeamState> SpearBeamState;

public struct BeamState
{
    public bool isHorizontal;
    public bool addPoles;
    public bool[] wasBeam;
    public bool hasBeamState;
}

```

And initialize your struct in the constructor or corresponding method, and Add to Dictionary.

```

public static void PlaceRoomPatch(On.Spear.orig_PlaceInRoom orig, Spear spear, Room placeRoom)
{
    orig.Invoke(spear, placeRoom);

    #region myCode
    if (spear is ExplosiveSpear) { return; }
    BeamState beamState = new BeamState();
    if (SpearBeamState.TryGetValue(spear, out beamState)) { return; } // already existing state

    beamState.wasBeam = new bool[3];
    if (spear.abstractSpear.stuckInWall)
    {
        beamState.hasBeamState = true;
        // more codes...
    }

    SpearBeamState.Add(spear, beamState); //add value to Dictionary
    #endregion
}

```

Whenever you need to set the value again, Remove and Add from Dictionary.

```

public static void ChangeModePatch(On.Spear.orig_ChangeMode orig, Spear spear, Weapon.Mode newMode)
{
    orig.Invoke(spear, newMode);

    #region myCode
    if (newMode == Weapon.Mode.StuckInWall)
    {
        BeamState beamState;
        if (!(spear is ExplosiveSpear) && SpearBeamState.TryGetValue(spear, out beamState))
        { // You can just use SpearBeamState[spear] like Array but just to be safe.
            // codes...
            SpearBeamState.Remove(spear);
            SpearBeamState.Add(spear, beamState);
        }
    }
    #endregion
}

```

6. EnumExtender

EnumExtender is a tool mod by bee, which allows you to add more items in `enums` much easier.

To use this, first, you need to download EnumExtender from RainDB-Tools tab, and put it in your Mods folder. You don't necessarily have to *Reference* EnumExtender unless you are doing more advanced stuff. Like Config Machine, the users are also required to have & enable EnumExtender to run your mod properly.

First, create a public static class with a name starting with `EnumExt_`. EnumExtender will automatically detect this. Then create a field with the type of enum you want to extend, and item name for the field name.

```
public static class EnumExt_MyMod
{
    public static CreatureCommunities.CommunityID Vultures;

    public static SlugcatStats.Name YellowishWhite;
    // You can have multiple types of enum in a single class.
}
```

To access these new `enum` items, instead of using something like '`[enum].[item]`' just use the field. To compare it, use `if` statements.

```
public static SlugStatsPatch(On.SlugcatStats.orig_ctor orig, SlugcatStats stats, int slugcatNumber, bool
    malnourished)
{
    orig.Invoke(stats, slugcatNumber, malnourished);
    if (stats.name == EnumExt_MyMod.YellowishWhite)
    {
        stats.runspeedFac = 1.2f;
        stats.poleClimbSpeedFac = 1.25f;
        stats.corridorClimbSpeedFac = 1.2f;
    }
}
```

You can also use `switch case` with a Dictionary.

```
if (TypeToIndex == null)
{
    TypeToIndex = new Dictionary<SlugcatStats.Name, int>()
    {
        { EnumExt_MyMod.YellowishWhite, 0 },
        { EnumExt_MyMod.WhitishYellow, 1 }
    };
}
if (TypeToIndex.TryGetValue(stats.name, out int i))
{
    switch (i)
    {
        case 0:
            stats.runspeedFac = 1.2f;
            stats.poleClimbSpeedFac = 1.25f;
            stats.corridorClimbSpeedFac = 1.2f; break;
        case 1:
            stats.runspeedFac = 0.8f;
            stats.poleClimbSpeedFac = 0.75f;
            stats.corridorClimbSpeedFac = 0.8f; break;
    }
}
```

And... that's about it! This mod is really easy to use and understand, but as many things in Rain World is set in `enums`, having an ability to edit them gives you so much potential. See [<Extra 4>](#) to read more about it.

To check if the user actually has `EnumExtender`, by the way, create a `bool` like this. You can then either disable features or throw message with `LogError` to track this issue easier.

```
public static bool HasEnumExt => (int)EnumExt_MyMod.YellowishWhite < 1;
// Without EnumExtender, they won't be initialized and will stay 0.
// The reason we don't check this in OnEnable and using a property is that EnumExtender's initialization
// happens after loading all mods.
```

There is another tool mod by bee that's worth checking out, which is `AutoUpdate`, but since the nature of this tool requires authorization from bee themselves (which means you'll *have to* ask bee what to do with it) so I will just link the tutorial by the author ([Link](#)).

7. Config Screen

To support Config Machine, you need two things. First, a class inheriting `OptionInterface`. and second, `PartialityMod.LoadOI()` that returns your `OptionInterface`.

First download Config Machine and put it in your Reference folder (with XML file in the same folder) and Reference `ConfigMachine.dll`. Then create a new cs file and create a class inheriting `OptionInterface`. We will use Colorfoot mod as an example in our tutorial.

```
using UnityEngine;
using RWCustom;
using OptionalUI;

public class ColorOption : OptionInterface
{
    public ColorOption() : base (ColorMod.instance) // PartialityMod instance
    {
    }

    public override void Initialize()
    {
        base.Initialize();
    }

    public override void Update(float dt)
    {
        base.Update(dt);
    }

    public override void ConfigOnChange()
    {
    }
}
```

`Update` runs every frame. `dt` represents `deltaTime` which is useful if you are doing graphical changes. `Initialize` runs when as you enter IntroRoll(to load default config) or Mod Config, and this is where you design your config screen. `ConfigOnChange` runs every time Config is changed, including Rain World initialization. So it is best to put your configurable variable initialization codes in here.

Next is to add `LoadOI()` in your `PartialityMod` class.

```
public static OptionInterface LoadOI()
{
    return new ColorOption();
}
```

Now your mod supports Config Machine. It is also now dependent to Config Machine.

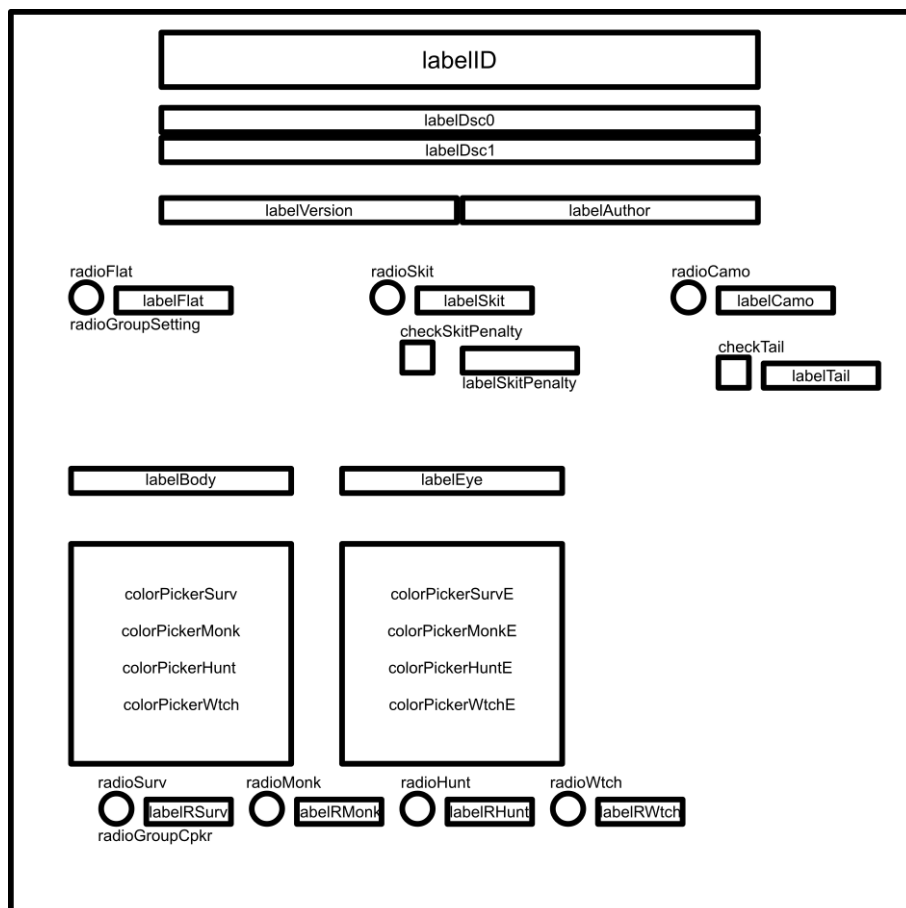
Let's design Config screen then! Config screen can consist of multiple tabs (max 20), each tab sized 600 * 600 pixels. How many tabs do you need?

If you've decided, let's initialize that! In our example, Colorfoot mod uses two OpTabs.

```
public override void Initialize()
{
    base.Initialize(); //this initializes tabs to null-- put it first

    this.Tabs = new OpTab[2]; //Create an Array of tabs
    this.Tabs[0] = new OpTab("Config");
    this.Tabs[1] = new OpTab("Credit");
}
```

Let's design GUI! I *strongly* suggest using Inkscape([Link](#)), create a file with 600x600 pixel sized, then enable [View]-[Canvas Orientation]-[Flip Vertically]. However, any computer / analogue tool that you like is fine.



The image above is a diagram of Colorfoot mod GUI in Tabs[0]. Once you've designed your GUI like this, go back to your mod project.

How do we add an UIelement? First, define UIelement with a nice name. Even if you never had any experience with Visual Basic, you can see how I named the elements. If it's an OpLabel,

labelSomething; if it is an OpImage, imageSomething. This isn't required but recommended for your convenience.



In this case, the OpLabel where the ModID will be written is spaced/sized like above. When you make a new instance of OpLabel you need to fill these:

```
public OpLabel(Vector2 pos, Vector2 size, string text = "TEXT", FLabelAlignment alignment =
    FLabelAlignment.Center, bool bigText = false);
```

So we need left bottom position, which is (100, 550), size (400, 40), text ("Colorfoot Mod"). The rest is not mandatory but I will set bigText = true because this is a title.

Next, we need to add this UIElement to corresponding OpTab. Use AddItems method to do this. That's all! Now your UIElement will be visible in Mod Config menu! You can compile and test that right now!

```
public OpLabel labelID; // If you don't need to access this after Initialization, it's not necessary to have
    the field outside Initialize method.

// In Initialize(), after initializing this.Tabs[0]
labelID = new OpLabel(new Vector2(100f, 550f), new Vector2(400f, 40f), "Colorfoot Mod",
    FLabelAlignment.Center, true); // Initializing OpLabel
Tabs[0].AddItems(labelID); // ...then add it to OpTab
```

This applies to every UIElements, so repeat. Some UIElements don't require size; those usually have a fixed size, and you can check them with their summary (if you have XML file).

Now, how do you access your config? Let us say your mod have a configurable bool value. So you add OpCheckBox like this:

```
checkSkitPenalty = new OpCheckBox(new Vector2(260f, 360f), "MyModSkitPenalty", true)
    { description = "If this is enabled, Skittlecat will become much more noticable to predators." };
// Any UIElement can have description, which displays at the bottom when you hover your mouse over.
// It is advised to mark every key of UIconfigs with your mod's unique ID because ConfigMachine can crash if
    multiple mods have the same key due to major oversight
Tabs[0].AddItems(checkSkitPenalty);
```

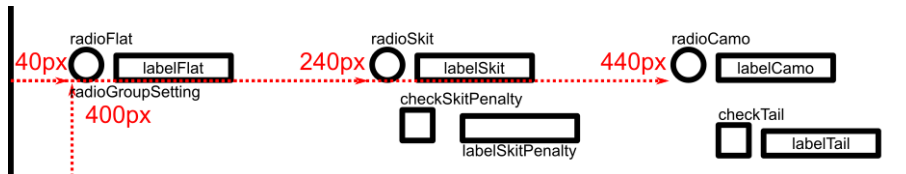
All UIconfig requires two extra attributes than just UIElement: key, and defaultValue. The key is a unique string to access this config value(setting key Empty/null or something that starts with _ will make it cosmetic, and won't be saved), and defaultValue is the value will be original if it wasn't already set or get reset. (You can keep test while adding UIconfig step by step, by the way. whether you add/remove UIconfigs or not, Config Machine retains backward compatibility)

OpRadioButton(Only one button out of group can be chosen at a time) is a bit special: it looks like a configurable setting, but actually, it is not OpConfig. To use OpRadioButton, you need to create

OpRadioButtonGroup.

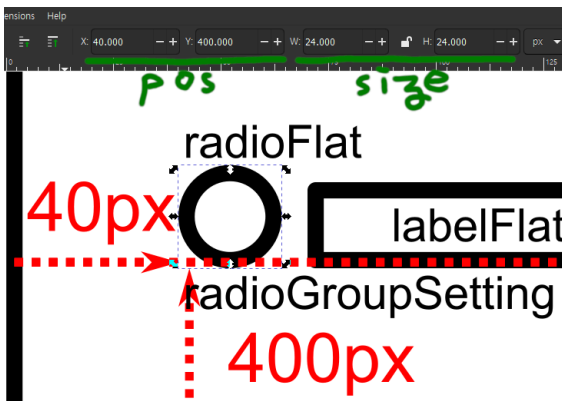
First, you initialize OpRadioButtonGroup. This is the actual UICongfig, and outputs int. (Of course, you need to use int.Parse since all config are kept as string) Then we add OpRadioButtonGroup to the tab. It is not visible, but necessary to be saved and used.

```
radioGroupSetting = new OpRadioButtonGroup("MyModSetting", 0);
Tabs[0].AddItems(radioGroupSetting);
```



Now initialize OpRadioButtons, then add these to the group. The order of OpRadioButtons is important, as they will each represent 0, 1, 2, and so on. They will automatically be added to the same OpTab as the OpRadioButtonGroup.

If you were using Inkscape and its [Flip Vertically] option, the object's X and Y coordinate and their size matches with what you need for their constructors.



```
radioGroupSetting.SetButtons(new OpRadioButton[]
{
    new OpRadioButton(40f, 400f) { description = "Flat Colored Slugcat" },
    new OpRadioButton(240f, 400f) { description = "Rainbow Colored Slugcat" },
    new OpRadioButton(440f, 400f) { description = "Camouflaging Slugcat" }
});
```

Config values are saved in strings, in config Dictionary. You can access it with config[key] anytime, but to minimize the access, override ConfigOnChange(). Your mod's configurations will be automatically saved and loaded by Config Machine so you don't need to worry about that part.

```
public override void ConfigOnChange()
{
    // This method runs every time config Dictionary is changed: When CM Loads default/saved config in
    // IntroRoll, or saving new configuration in Mod Config
    base.ConfigOnChange();
    MyMod.config.configMachine = true;
    // create a struct to save configurables, and put everything in there. This way you can easily build two
    // separate version(Config Machine version/non-configurable version) easily
    MyMod.config.setting = int.Parse(config["MyModSetting"]); // Parsing int
    MyMod.config.skitPenalty = config["MyModSkitPenalty"] == "true"; // Parsing bool
    MyMod.config.survivorBody = OpColorPicker.HexToColor(config["MyModSurvivorBody"]); // Parsing Hex to
    // Unity Color
}
```

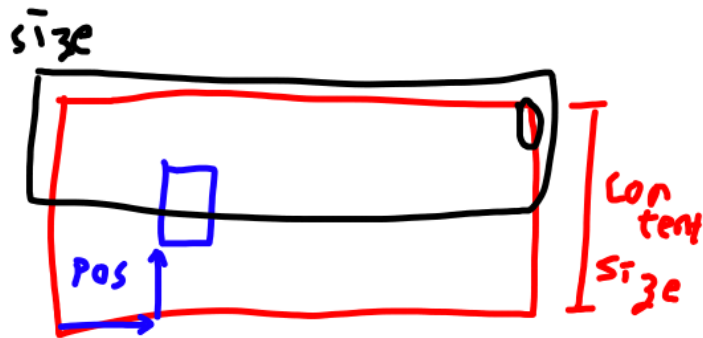
There is another special type of UIelement, which is UITrigger. OpSimpleButton is the most representative (and only, as of Config Machine 1.2) one of these. Adding these to config screen is similar to UICongfig, except its signal does not have to be unique. When the user trigger these, it will call OptionInterface.Signal(UITrigger trigger, string signal), which you can override and

write the code you want to execute.

OpScrollBox is another special UIelement that's added in Config Machine 1.4, by the contribution of Slime_Cubed. It acts like a custom OpTab that can be scrolled. There's two way of initializing this. As a small window inside OpTab, or as the size of OpTab that practically replaces it.

```
// For mini-sized OpScrollBox
OpScrollBox sbv = new OpScrollBox(new Vector2(50f, 50f), new Vector2(500f, 100f), 400f, false);
Tabs[1].AddItems(sbv);
// Use OpScrollBox.AddItems instead of OpTab.AddItems.
sbv.AddItems(new OpKeyBinder(new Vector2(50f, 350f), new Vector2(80f, 24f), this.mod, "_", "", false));
```

The positioning of UIelements inside OpScrollBox is similar to how you do with OpTab. The origin is bottom left of OpScrollBox's content canvas, and you put your UIelements with the relative position from the origin.



The other way of using OpScrollBox is that constructing it with OpTab and practically replacing that OpTab.

```
// For canvas-sized OpScrollBox
OpScrollBox sb = new OpScrollBox(Tabs[2], 2400f, false);
// Use OpScrollBox.AddItems instead of OpTab.AddItems.
OpColorPicker cpk0 = new OpColorPicker(new Vector2(100f, 100f), "_TEST", "CCCCC") { colorEdge = Color.red };
// This is Lazier method of initializing OpLabel. Some provided UIElements like OpRadioButton has Lazier ctor.
sb.AddItems(cpk0, new OpLabel(300f, 100f, "Red Colorpicker") { bumpBehav = cpk0.bumpBehav, color = Color.red });
// Appending BumpBehaviour to OpLabel.bumpBehav makes its colour react with the UIconfig/UITrigger
```

You can also design your own UIelement/UIconfig by inheriting them if it is necessary. If that is required, check the summary of virtual methods. In ctor, when init is false (which means this is in IntroRoll and not in Mod Config) then you *should not* initialize objects that require this.menu, owner, subObjects, nextSelectable, and myContainer. And, you must write Hide(), Show(), Unload() at least to make a proper UIelement. UIconfig also has greyedOut bool, which locks interaction with that configurable if it is true.

If you have done something wrong, Config Machine will display a bluescreen. It may show expected logical error so you might spot the problem easier.

You can also use Config Machine to support translation, whether it's UI or in-game message/dialogues. See <[Extra 5](#)> for the tutorial.

Extra 1: Accessing private value in Hooking

This tutorial recommends you to use PublicityStunt, but using this to accessing private/protected values will make it dependent to PublicityStunt. While you can mark your mod that it requires PublicityStunt, you can avoid this dependency.

But first, check if your mod breaks without PublicityStunt with exceptions like `FieldAccessException` OR `MethodAccessException`, which indicates your mod is dependent to PublicityStunt. If your mod does not break then you won't need to worry about all these. However, if your mod is confirmed to have a dependency of PublicityStunt, (while you can just publish it with dependency) you can either take one of these methods.

First, the old method is not using PublicityStunt at all but use `Reflection`. Instead of referencing modified `Assembly-CSharp.dll`, run Integrity Check and grab vanilla `Assembly-CSharp.dll`. This will help you to know where you should use `Reflection`. `Reflection` is generally slow and should be avoided in normal circumstances, but this is not just Rain World modding thing so there are plenty of resources about this online.

This is the example of accessing a field.

```
using System.Reflection;

public const BindingFlags flagInst = BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Instance;
// Having this sitting somewhere helps if you're planning to use Reflection a lot.
// Use BindingFlags.Static instead of Instance if you're accessing something static.

([returnType])typeof([class]).GetField("fieldName", flagInst).GetValue([class instance]); // getter
typeof([class]).GetField("fieldName", flagInst).SetValue([class instance], [newValue]); // setter
```

Second, the newer method is using bee's code. Create a cs file in your mod project, and copy paste the code by bee from this link below:

<http://beestuff.pythonanywhere.com/personal/partiality/publicitystunt>

You still need a modified `Assembly-CSharp.dll` by PublicityStunt in Reference to compile your mod, but the users will not need that to run your mod. That just works magically.

Extra 2: Making a Partiality Patch

This tutorial taught you how to make Partiality Mod, with Hooking, but there is another way to mod Rain World other than direct dnSpy modding(which is not completely legal and has little to zero compatibility, so this tutorial *will not* cover that): Partiality Patch.

You do *not* create PartialityMod class in your project. (Partiality *does* load PartialityMod class even if your mod is Partiality Patch, but it is not intended behaviour) You also do not reference HOOK-Assembly-CSharp.dll, but instead, you reference MonoMod.exe found in Partiality Launcher's bin folder.

There is one advantage. The first is that it's much easier to add non-static fields to pre-existing classes. You could also access private/protected field/method without Reflection, and edit enum much easily, but now you can use PublicityStunt, EnumExtender to avoid this issue. It is said that this is much easier to understand for someone who's not good at C#, but the disadvantages are too significant.

The main disadvantage of Partiality Patch is, that multiple Partiality Patches that edit the same method won't be compatible. Not having PartialityMod would also prevent supporting Config Machine or AutoUpdate. But let's get into how this is done, because Drought is made with this and it is at least worth mentioning.

Unlike Hooking, you do need to create a class for each class you want to modify, then patch constructor even if you won't modify it. In this example, we will patch PlayerGraphics. In your project, it'd look like this.

```
// Using this is NOT recommended, but at least one MonoModPatch is required to let Partiality Launcher know
// that this is, indeed, Partiality Patch
// If you're patching a class within a namespace, it'd be like "namespace.class"
[MonoModPatch("global::PlayerGraphics")]
public class patch_PlayerGraphics : PlayerGraphics
{
    [MonoModIgnore]
    public patch_PlayerGraphics (PhysicalObject ow) : base (ow)
    {
    } // This just removes compile error. It doesn't do anything other than that.

    public extern void orig_ctor (PhysicalObject ow); // This represents original constructor.
    [MonoModConstructor]
    public void ctor(PhysicalObject ow) { orig_ctor(ow); }
    // This is modified constructor, which just calls original constructor. However, having this is
    // recommended even if you don't actually need to modify constructor.
}
```

Patching method is the same as the constructor, just without [MonoModConstructor]. To access private fields or methods, add a public field/method that has the same name and stuff, and add [MonoModIgnore] on top to prevent it actually being overwritten.

The example below makes slugcat alternate its colour in a rainbow passion. For a more detailed tutorial on modifying slugcat's graphic by adding sprites, continue to <[Extra 3](#)>.

```

public extern void orig_DrawSprites (RoomCamera.SpriteLeaser sLeaser, RoomCamera rCam, float timeStacker,
Vector2 camPos ); // Represents original method.
public new void DrawSprites (RoomCamera.SpriteLeaser sLeaser, RoomCamera rCam, float timeStacker, Vector2
camPos )
{
    orig_DrawSprites (sLeaser, rCam, timeStacker, camPos); // Calls original method.
    hue += timeStacker * 0.02f * this.player.aerobicLevel;
    if (hue > 1f){ hue -= 1f; }
    for (int i = 0; i < sLeaser.sprites.Length; i++)
    {
        if (i == 9){ continue; }
        sLeaser.sprites[i].color = Custom.HSL2RGB(hue, 1f, 0.5f);
    }
}

public float hue = 0f; // The only advantage of Patch is you can add non-static variable like this easily
[MonoModIgnore]
public Player player;
// player is private in vanilla, which would cause a compile error.
// So we make a public field that has the same type and name from vanilla and add [MonoModIgnore] so we
don't actually patch it.

```

If your mod has at least one [PartialityPatch], Partiality Launcher will automatically recognize your mod as Partiality Patch and do the patching for you, so at least you don't need to worry about forgetting to patch something.

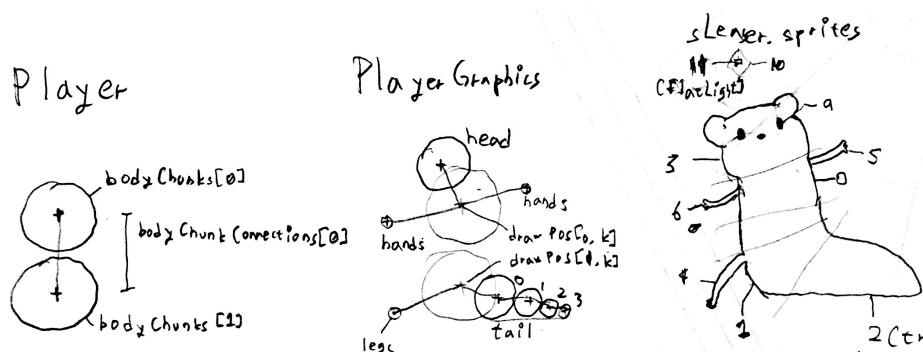
However, it is important to clarify whether your mod is Partiality Mod or Partiality Patch when you release it. BepInEx's Partiality Wrapper requires them to be placed in separate method, so the users must know if your mod is Partiality Patch.

Extra 3: Modifying Slugcat's Graphic

Since this is what the most of new modders want to do, adding own dank slugcat OC in pixelated form, and it is a good practice to step into modding graphics in Rain World, I will have this Extra dedicated to that.

Along with C#, you need to have some knowledge of vector maths([Manual Link](#)) and the difference between position vectors and direction vectors. If you want to dig deeper, study on trigonometrical functions too.

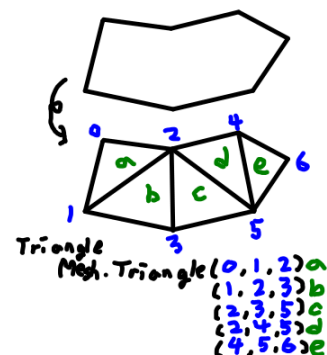
So we would need to edit `PlayerGraphics` to do this. More specifically, we will hook `InitiateSprites`, `DrawSprites` to begin with. In this example, we will look into Lancer's iconic horn, which is a `TriangleMesh`.



Slugcat in vanilla has a total of 11 sprites as we all know. However, that's not always true with other slugcat editing mods, so we can't simply hardcode that and call it a day. In this example of adding a horn, we need to add 1 more sprite, and we need to know the index of our new sprite since it may not always be 12th. First, we make a field to hold this varying index.

```
public static int[] horn;  
// This will keep the index of horn sprite. Initialize this with 4(max player num) indexes in OnEnable
```

Now we will patch `InitiateSprites` to, well, initiate our new sprite. We could use `FSprite` and add a sprite image. However, that's a bit tedious for such a simple graphic like a triangle in this example. So we'll use `TriangleMesh`, which is something you should learn sooner or later if you're planning to dive deep into custom graphics. To use this, give indexes to the points of the polygon you want to draw, and triangulate. See how many triangles it is consisted with, then make `TriangleMesh.Triangle` accordingly. You should keep the indexes of points in your mind, as you need them to update their position later.



```

public static void InitSprPatch(On.PlayerGraphics.orig_InitiateSprites orig, PlayerGraphics instance,
    RoomCamera.SpriteLeaser sLeaser, RoomCamera rCam)
{
    int slugcat = (instance.owner as Player).playerState.playerNumber;
    horn[slugcat] = int.MaxValue;
    orig.Invoke(instance, sLeaser, rCam);

    horn[slugcat] = sLeaser.sprites.Length; // Saving the index for horn
    Array.Resize(ref sLeaser.sprites, sLeaser.sprites.Length + 1); // Increasing sLeaser.sprites without
    // destroying other mods' modifications
    TriangleMesh.Triangle[] tris = new TriangleMesh.Triangle[]
    { new TriangleMesh.Triangle(0, 1, 2); }; // Check the diagram above to see how to initialize this.
    TriangleMesh triangleMesh = new TriangleMesh("Futile_White", tris, false, false);
    sLeaser.sprites[horn[slugcat]] = triangleMesh;

    instance.AddToContainer(sLeaser, rCam, null);
    // AddToContainer method in vanilla automatically adds your new sprites in Midground, but in front of
    // other slugcat's sprites since this has the last index. You can either let that happen or hook
    // AddToContainer too to change the Layer of your new sprites
}

```

Now we'll hook DrawSprites to update them, but first, we need a clear image for where the new sprite should be, and how should it look. In this example, the horn is an isosceles triangle on slugcat's head. The horn's height is 8 pixels and its direction is away from slugcat's body. The width is 4 pixels and it is perpendicular to its height since this is an isosceles triangle.

To draw this, we need to calculate where the three points of this triangle should be. PlayerGraphics has some coordinates we can work with, such as drawPositions(which is where two BodyChunks are), and head(which is where the head a.k.a. the first BodyChunk is). With those, we can find where is the head and which direction is up for slugcat.



```

Vector2 draw1 = Vector2.Lerp(instance.drawPositions[1, 1], instance.drawPositions[1, 0], timeStacker); //
    Draw Position of the second BodyChunk, which is Slugcat's butt
Vector2 head = Vector2.Lerp(instance.head.lastPos, instance.head.pos, timeStacker); // Draw Position of the
    first BodyChunk, which is Slugcat's head

Vector2 tip = head + Custom.DirVec(draw1, head) * 8f;
// Custom.DirVec gives a direction vector from param1 to param2 with the magnitude of 1f.
Vector2 thicc = Custom.PerpendicularVector(head, tip);
// Custom.PerpendicularVector gives a direction vector that is perpendicular from the vector that starts
    from param1 to param2.
// draw1, head, tip are position vectors, and thicc is a direction vector. If we call position vector P and
    direction vector V:
// (V + V).normalized = V, (V - V).normalized = V, (P - P).normalized = V (Not all V have to have magnitude
    of 1f, but it helps reducing confusion)
// P + V = P, P - V = P; P + P = Error! (This combines two offsets, resulting a corrupted vector)

```

After you determine the position vector of each vertex, subtract that with camPos to offset with current camera position. Yes, in Rain World, 'camera' doesn't move, but the whole world just shifts.

```

Vector2 dir = (tip - head) * 0.6f; // This is a direction vector for an offset from head to the start of
horn.
int slugcat = (instance.owner as Player).playerState.playerNumber;

(sLeaser.sprites[horn[slugcat]] as TriangleMesh).MoveVertice(0, dir + tip - camPos);
(sLeaser.sprites[horn[slugcat]] as TriangleMesh).MoveVertice(1, dir + head - thicc * 2f - camPos);
(sLeaser.sprites[horn[slugcat]] as TriangleMesh).MoveVertice(2, dir + head + thicc * 2f - camPos);

```

This is very basic overview on PlayerGraphics and TriangleMesh, but you'll be able to step farther with other graphics or more complicated mesh calculation.



Extra 4: Adding Creatures using EnumExt

While modifying pre-existing creatures is a toy you can easily play with, adding a creature is where the real fun begins, as long as you have a weird idea of fun. I assume you do have that weird idea of fun since you probably loved playing Rain World so much that decided to make a mod of it.

EnumExtender is a powerful tool that can extend any enum, and as types of creatures in Rain World is stored in enum `CreatureTemplate.Type`, while this is *not the only way* of using this tool, it is very helpful for adding creatures. However, you'd need to know where to edit and what to add to actually pull this off.

Before everything, let's use EnumExtender to extend `CreatureTemplate.Type`.

```
public static class EnumExt_Rainbow
{ public static CreatureTemplate.Type Hornest; }
```

First, you'd need to create a class for the your Creature Realized. In this example, I used normal Creature, but there're other base classes like `AirBreatherCreature`, `InsectoidCreature`.

```
public class Hornest : Creature
{
    public Hornest(AbstractCreature abstractCreature, World world) : base(abstractCreature, world)
    {
        // BodyChunk is a physical Hit-circle that's used for pretty much most of physical calculations
        this.bodyChunks = new BodyChunk[2];
        this.bodyChunks[0] = new BodyChunk(this, 0, new Vector2(0f, 0f), 30f, 5f);
        this.bodyChunks[1] = new BodyChunk(this, 1, new Vector2(0f, -50f), 0.1f, 5f);
        // BodyChunkConnection holds two BodyChunks in a certain distance, with different forces(Normal, Pull, Push)
        this.bodyChunkConnections = new BodyChunkConnection[1];
        this.bodyChunkConnections[0] = new BodyChunkConnection(this.bodyChunks[0], this.bodyChunks[1], 100f, BodyChunkConnection.Type.Pull, 0.8f, -1f);

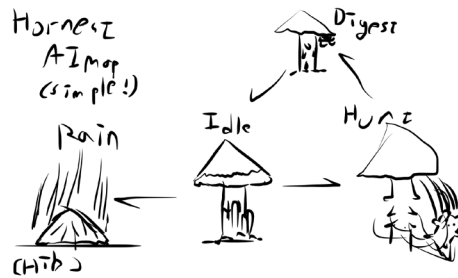
        // These value are from Scavenger, but do experiment.
        base.airFriction = 0.999f; // 1.0f for no friction, Lower it for steeper arc of parabola.
        base.gravity = 0.9f; // Every frame, this gravity will be added to BodyChunks' vertical velocity
        this.bounce = 0.1f; // When this hits the terrain, it will bounce with this multiplier
        this.surfaceFriction = 0.4f; // This doesn't matter with conscious creatures, but does matter with unconscious creatures/items.
        this.collisionLayer = 1; // creatures are usually 1, some items are 0 and 2.
        base.waterFriction = 0.96f;
        base.buoyancy = 0.95f; // This gets added to vertical velocity depends on how much this is submerged
    }
    public override void InitiateGraphicsModule()
    {
        if (base.graphicsModule == null) { base.graphicsModule = new HornestGraphics(this); }
    }
    public override void Update(bool eu)
    {
        base.Update(eu); // Don't forget to have base.Update to run physical calculations!
        if (this.room == null) { return; } // This usually prevents unwanted NullReferenceException.

        // More Codes! But here's some example you can do in here.
        if (this.Consious)
        { base.abstractCreature.abstractAI.RealAI.Update(); // You need to update AI manually. }
        else
        {
            if (this.grasps[0] != null) { this.ReleaseGrasp(0); }
            // Obvious behaviour in hindsight but this is something you need to specify.
        }
    }
}
```

Now we will need GraphicsModule to display this. For items you can use interface IDrawable to your item class, which functions mostly the same.

```
public class HornestGraphics : GraphicsModule
{
    public HornestGraphics(PhysicalObject ow) : base(ow, false)
    {
        // Codes!
    }
    public override void InitiateSprites(RoomCamera.SpriteLeaser sLeaser, RoomCamera rCam)
    {
        // Initiate Sprites!
        this.AddToContainer(sLeaser, rCam, null);
        base.InitiateSprites(sLeaser, rCam);
    }
    public override void AddToContainer(RoomCamera.SpriteLeaser sLeaser, RoomCamera rCam, FContainer newContainer)
    {
        // Add to FContainer so you can actually see them!
        base.AddToContainer(sLeaser, rCam, newContainer);
    }
    public override void DrawSprites(RoomCamera.SpriteLeaser sLeaser, RoomCamera rCam, float timeStacker, Vector2 camPos)
    {
        base.DrawSprites(sLeaser, rCam, timeStacker, camPos);
        // Update Sprites! This usually requires you to know vector maths and trigonometrical functions.
    }
    public override void ApplyPalette(RoomCamera.SpriteLeaser sLeaser, RoomCamera rCam, RoomPalette palette)
    {
        base.ApplyPalette(sLeaser, rCam, palette);
        // Color stuff!
    }
}
```

For AI, the AI of the most creatures in Rain World uses State Space (google if you don't know what this is). For example, Lizard in default is Idle. However, Lizard can track creatures and situation surrounding it, and change its state(LizardAI.Behavior) when new priority comes up. If there's a dire threat Lizard will enter Flee state, EscapeRain state when it's about to rain, Hunt state when it spotted a prey. Even the simplest creatures' AI would be better to deal with State Space so draw a diagram before getting into making an AI. The diagram on the right is one example.



```
public class HornestAI : ArtificialIntelligence
{
    public HornestAI(AbstractCreature creature, World world) : base(creature, world)
    {
        this.nest = (creature.realizedCreature as Hornest);
        this.nest.AI = this;
        base.AddModule(new StandardPather(this, world, creature)); // For pathfinding
        base.pathFinder.stepsPerFrame = 10;
        base.AddModule(new Tracker(this, 10, 5, 1000, 2f, 5, 10, 50)); // To track stuffs nearby
        base.AddModule(new RainTracker(this)); // To track rain timer
        base.AddModule(new PreyTracker(this, 3, 1.5f, 5f, 20f, 0.5f)); // To track prey specifically

        // Comparing Priority
        base.AddModule(new UtilityComparer(this));
        base.utilityComparer.AddComparedModule(base.rainTracker, null, 0.8f, 1.1f);
        FloatTween smoother = new FloatTween.FloatTweenBasic(FloatTween.TweenType.Tick, 0.01f);
        base.utilityComparer.AddComparedModule(base.preyTracker, smoother, 1f, 1.1f);
        // Experiment with these priorities and settings

        this.behavior = HornestAI.Behavior.Idle; // State Space.
        //this.tracker.visualize = true; // This is useful to check if your creature is tracking well
    }
}
```

```

public Hornest nest; // Having this instead of just using owner is useful in many case

public enum Behavior
{ // These are several states this creature can have
    Idle,
    Anticipate,
    Release,
    Digest,
    EscapeRain
}

public HornestAI.Behavior behavior;

public int digesting; public float angry;
public override void Update()
{
    base.Update();
    if (this.nest.room == null) { return; }

    // The following is an example of changing States inbetween.
    AIModule aimodule = base.utilityComparer.HighestUtilityModule();
    if (digesting > 0)
    {
        digesting--;
        this.behavior = HornestAI.Behavior.Digest;
        if (digesting == 0) { this.nest.AddFruit(); }
    }
    else if (aimodule != null && this.behavior != HornestAI.Behavior.Release)
    {
        if (aimodule is RainTracker) { this.behavior = HornestAI.Behavior.EscapeRain; }
        else if (aimodule is PreyTracker) { this.behavior = HornestAI.Behavior.Anticipate; }
    }
    if (base.utilityComparer.HighestUtility() < 0.011f && digesting == 0)
    { this.behavior = Behavior.Idle; }

    if (this.nest.Consious && this.angry >= 1f && this.behavior == Behavior.Anticipate &&
        this.targetCreature != null)
    {
        this.angry = 1f;
        this.behavior = HornestAI.Behavior.Release;
        if (this.nest.releaseBeesDelay == 0) { this.nest.ReleaseBees(); }
    }

    // This part is where actual AI stuffs happen but is mostly omitted out except the parts that explain
    // State Space
    switch (this.behavior)
    {
        case Behavior.Idle:
            if (this.angry > 0f) { this.angry = Mathf.Max(0f, this.angry - 0.005f); }
            break;
        case Behavior.Anticipate:
            angry += 0.02f;
            break;
        case Behavior.Release:
            // Grabbing victim and setting AI.digest happens in Creature's Update
            break;
        case Behavior.Digest:
            this.angry = Mathf.Max(0f, this.angry - 0.04f);
            break;
        case Behavior.EscapeRain:
            this.angry = 0f;
            break;
    }
}
}

```

You don't have to be exceptionally good in coding for this; you just need to be creative, and are able to express your algorithm in codes.

Now we're going to edit `StaticWorld` so it also has the basic information about your new creature. This modification needs to be done after all the mods have been loaded (a.k.a. `EnumExtender` did its job), but before the actual game begins, and only once. You can try hooking `RainWorld.Start`, but you shouldn't put this script in `OnEnable` as `EnumExtender` isn't ready that early.

```
List<CreatureTemplate> vlist = new List<CreatureTemplate>();
// Load all creature from StaticWorld; this may include custom creatures from other mods
for (int i = 0; i < StaticWorld.creatureTemplates.Length; i++)
{ vlist.Add(StaticWorld.creatureTemplates[i]); }

List<TileTypeResistance> ttr = new List<TileTypeResistance>();
List<TileConnectionResistance> tcr = new List<TileConnectionResistance>();
// To determine movement pattern; experiment with this according to your creature
ttr.Add(new TileTypeResistance(AITile.Accessibility.Air, 1f, PathCost.Legality.Allowed));
ttr.Add(new TileTypeResistance(AITile.Accessibility.OffScreen, 1f, PathCost.Legality.Allowed));
tcr.Add(new TileConnectionResistance(MovementConnection.MovementType.Standard, 1f,
    PathCost.Legality.Allowed));
tcr.Add(new TileConnectionResistance(MovementConnection.MovementType.OutsideRoom, 1f,
    PathCost.Legality.Allowed));
tcr.Add(new TileConnectionResistance(MovementConnection.MovementType.SkyHighway, 1f,
    PathCost.Legality.Allowed));
tcr.Add(new TileConnectionResistance(MovementConnection.MovementType.OffScreenMovement, 1f,
    PathCost.Legality.Allowed));
tcr.Add(new TileConnectionResistance(MovementConnection.MovementType.BetweenRooms, 10f,
    PathCost.Legality.Allowed));

// Add our custom creature
vlist.Add(new CreatureTemplate(EnumExt_Rainbow.Hornest, null, ttr, tcr, new
    CreatureTemplate.Relationship(CreatureTemplate.Relationship.Type.Ignores, 0f)));
vlist[vlist.Count - 1].name = "Hornest";
vlist[vlist.Count - 1].baseDamageResistance = 100f;
//vlist[vlist.Count - 1].saveCreature = true;
vlist[vlist.Count - 1].AI = true; // check true if your creature has realized AI
vlist[vlist.Count - 1].grasps = 1; // how many things it can grab
vlist[vlist.Count - 1].visualRadius = 800f;
vlist[vlist.Count - 1].dangerousToPlayer = 0.5f; // set this to make Iggy warn this creature
vlist[vlist.Count - 1].countsAsAKill = 1; // set this if this creature can kill other creatures
// change more attributes as your creature needs

// Add this back to StaticWorld
StaticWorld.creatureTemplates = new CreatureTemplate[vlist.Count];
for (int i = 0; i < vlist.Count; i++)
{ StaticWorld.creatureTemplates[(int)vlist[i].type] = vlist[i]; }
```

You'll also need to establish `CreatureTemplate.Relationships` with other creatures, which is another tedious thing but necessary to set how your new creature and other creatures react to each other.

Now let's add this creature in AbstractCreature, so it can be Abstractized and Realized.

```
public static void CtorPatch(On.AbstractCreature.orig_ctor orig, AbstractCreature self, World world,
    CreatureTemplate creatureTemplate, Creature realizedCreature, WorldCoordinate pos, EntityID ID)
{
    orig.Invoke(self, world, creatureTemplate, realizedCreature, pos, ID);
    CreatureTemplate.Type t = creatureTemplate.TopAncestor().type;
    if (t == EnumExt_Rainbow.Hornest)
    {
        // If your creature does not need abstractAI, skip this line
        self.abstractAI = new AbstractCreatureAI(world, self);
        // You do need any kind of CreatureState to let it function properly
        self.state = new NoHealthState(self); return;
    }
}

public static void RealizePatch(On.AbstractCreature.orig_Realize orig, AbstractCreature self)
{
    if (self.realizedCreature != null) { return; }
    CreatureTemplate.Type t = self.creatureTemplate.TopAncestor().type;
    // This allows creature to be Realized from its Abstract state.
    if (t == EnumExt_Rainbow.Hornest) { self.realizedCreature = new Hornest(self, self.world); goto
    InitAI; }
    orig.Invoke(self); return;
InitAI:
    // This is copy paste of original code since we're skipping it.
    self.InitiateAI();
    for (int i = 0; i < self.stuckObjects.Count; i++)
    {
        if (self.stuckObjects[i].A.realizedObject == null)
        { self.stuckObjects[i].A.Realize(); }
        if (self.stuckObjects[i].B.realizedObject == null)
        { self.stuckObjects[i].B.Realize(); }
    }
}

public static void InitiateAIPatch(On.AbstractCreature.orig_InitiateAI orig, AbstractCreature self)
{
    CreatureTemplate.Type t = self.creatureTemplate.TopAncestor().type;
    // If your creature doesn't have Real AI, skip this.
    if (t == EnumExt_Rainbow.Hornest) { self.abstractAI.RealAI = new HornestAI(self, self.world); return; }
    orig.Invoke(self);
}
```

Now your creature is working. Of course, spawning it is a completely different issue. To spawn them in Singleplayer, you need to edit world files so the dens can spawn your creature, then hook WorldLoader.CreatureTypeFromString so the game can load your creatures from the edited world files. (Editing world files means you need to disable checksum test)

```
public static Dictionary<string, int> newCritNames;
public static CreatureTemplate.Type? CreatureTypeFromStringPatch(On.WorldLoader.orig_CreatureTypeFromString
    orig, string s)
{
    if (!string.IsNullOrEmpty(s))
    {
        if (newCritNames == null)
        {
            newCritNames = new Dictionary<string, int>
            { { "hornest", 35 } };
        }
        if (newCritNames.TryGetValue(s.ToLower(), out int num))
        {
            switch (num)
            {
                case 35: return new CreatureTemplate.Type?(EnumExt_Rainbow.Hornest);
            }
        }
    }
    return orig.Invoke(s);
}
```

You can also add them to Sandbox mode with EnumExtender, but that's another big can of worms so I will just link this old tutorial ([Link](#)). It is aimed for dnSpy modding, but if you were able to follow me this far I'm sure you'll be able to use them in Hooking.

Another thing you can try, and much easier than adding creature, is adding new type of item. Add new item in `AbstractPhysicalObject.AbstractObjectType`, then Hook `AbstractPhysicalObject.Realize`.

```
public static void RealizePatch(On.AbstractPhysicalObject.orig_Realize orig, AbstractPhysicalObject self)
{
    orig.Invoke(self);
    if (self.realizedObject != null) { return; }
    if (self.type == EnumExt_Rainbow.Railgun)
    { self.realizedObject = new Railgun(self, self.world); goto StuckRealize; }

    StuckRealize:
    for (int i = 0; i < self.stuckObjects.Count; i++)
    {
        if (self.stuckObjects[i].A.realizedObject == null && self.stuckObjects[i].A != self)
        { self.stuckObjects[i].A.Realize(); }
        if (self.stuckObjects[i].B.realizedObject == null && self.stuckObjects[i].B != self)
        { self.stuckObjects[i].B.Realize(); }
    }
}
```

Extra 5: Supporting Translation using Config Machine

While you can support translation for your mod on your own, Config Machine also offers translation solution. So here's how it works: `OptionInterface` has a method called `Translate`, and when it's initially called, it will load specified(`transFile`) txt file from your assembly and make Dictionary depending on Rain World's language setting.

In `PartialityMod.LoadOI` method, store the `OptionInterface` instance in somewhere before returning it. I'll use Lancer Mod as an example.

```
public static LancerOption oi; // OptionInterface for Lancer
public static LancerOption LoadOI()
{
    oi = new LancerOption(); return oi;
}
```

Then make a static method that calls the translation method.

```
public static string Translate(string orig)
{
    if (oi != null) { return oi.Translate(orig); }
    return orig;
}
```

We need to create txt file that contains translation, too. Add txt file to your project, make sure that its encoding is UTF-8, then set it to be Embedded Resources for its compile setting. Then, in the constructor of your `OptionInterface`, set `transFile` to that resource name. If you have not put it in any folder, it's usually `<ProjectName>.<TxtFileName>.txt`, but you might get it wrong. If it's wrong when `OptionInterface.Translate` is initially called, Config Machine will log all the resources in your assembly in `exceptionLog.txt` so you can copy from it.

```
public LancerOption() : base(LancerMod.instance)
{
    this.transFile = "Lancer.Translation.txt";
    // if you do not know the resource name, just call Translate.
    // ConfigMachine will log all the resources in your assembly in exceptionLog.txt.
}
```

Now when you need to translate something, you can pass your string through it.

The format of the translation txt file for Config Machine is quite primitive. First, you add a keyword, then | for language separator, then language ID(`eng`, `ita`, `ger`, `spa`, `por`, (`kor`, `rus`)), \$ for another separator, then translation. If there is no translation, the translator will return the keyword, unless there is a translation for 'eng'. (In that case, English 'translation' will be used as default translation)

The following is some example for txt file.

```
// If the first two character is '//' , this line will be ignored.
Nightcat Horn Colour|kor$밤고양이 뽀 색갈
// If you have variables, avoid using partial sentences. Not every language has the same subject-verb-object
order.
Press <ThrowKey> to stab.|kor$<ThrowKey> 버튼을 눌러 찌르기.
// When translating the shorter phrase, the translation might be sensitive with context, but English one
does not. Duplicate key causes error(You can check exceptionLog.txt for duplicates), so you can set
'eng' translation for these cases.
Property_as of possession|eng$Property|kor$소유물
Property_as of quality|eng$Property|kor$속성
// For line-breaks, use \n. (In code, it'd be \\n) Each line represents a phrase, so you can't use actual
line-break in a single chunk.
LINE\nBREAK|kor$줄\n바꿈
```

And here are some examples in code. Yes, running every text through Translate method is tedious, but this is usually how it's done in Rain World or many other game.

```
labelCpkrRadio.text = LancerMod.Translate("Nightcat Horn Colour");
instance.room.game.cameras[0].hud.textPrompt.AddMessage(LancerMod.Translate("Press <ThrowKey> to
stab.").Replace("<ThrowKey>", k.ToString())); // k is variable in this example.
labelLineBreak.text = LancerMod.Translate("LINE\nBREAK");
```

These aren't the part of tutorial and more of tips from my personal experience. This does not just apply for making Rain World mods but to general, whenever you're making something that supports translation. As you're seeing, when you have variables in your sentence, do not use partial phrases and Frankenstein it later with `string.Concat`. Not every language has the same grammar order of subject-verb-object as English.

Also, I suggest leaving credits to the translators, since you're most likely getting volunteers from them. This is both good for showing gratitude to the translators, and keeping the quality of the translation. The lack of credits means a lack of responsibility. Finally, do not use machine translation. That usually hurts the user experience rather than enhancing it.

Extra 6: Preparing for VS Code

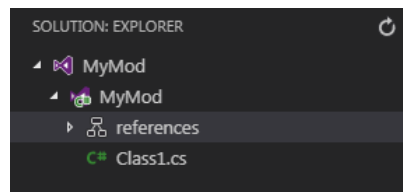
If you have technical difficulties, a broken computer, cannot install a program because the pc is not yours, OS is not Windows, need to work on mobile, the computer is so bad that can't use a heavy program like Visual Studio Community, then you have to use VS Code.

Copy `microsoftlib.dll` from Managed too when you are copying reference DLLs. Then download VS Code([Link](#)), and open it, then install these from Marketplace.

- C# from Microsoft (for Syntax Highlighting, IntelliSense)
- Visual Studio IntelliCode from Microsoft DevLabs (for enhancing IntelliSense)
- `vscode-solution-explorer` from Fernando Escobar (to build Class Library in VS Code)

Restart VS Code, and Open Folder to begin. Create a new folder with your desired name for the mod, then head to SOLUTION: EXPLORER, which will say 'No solution found'. Right-click it, create an Empty Solution with a name you want.

Now, right-click the new solution, and Add new project. For project type, choose 'class library', and for language, choose 'C#'. and type your mod name to create a project.



Then the trickiest part, we need to build this in .NET Framework 3.5, not netstandard2.0. We need to manually modify the csproj file to do this. Open your csproj file (use normal explorer to do this) that's automatically created. Also, we need to add references too, manually. so copy-paste these after `<Project Sdk="Microsoft.NET.Sdk">`, and modify `[PATH]` to your actual reference folder path.

```
<PropertyGroup>
  <TargetFrameworks>netstandard2.0;net350 </TargetFrameworks>
</PropertyGroup>
<PropertyGroup Condition=" '$(TargetFramework)' == 'net350' ">
  <FrameworkPathOverride>$(Mono)[PATH]\</FrameworkPathOverride>
</PropertyGroup>
<ItemGroup>
  <Reference Include="Assembly-CSharp">
    <HintPath>[PATH]\Assembly-
CSharp.dll</HintPath>
  </Reference>
  <Reference Include="ConfigMachine">
    <HintPath>[PATH]\ConfigMachine.dll</HintPath>
  </Reference>
  <Reference Include="HOOKS-Assembly-CSharp,
Version=0.0.0.0, Culture=neutral,
processorArchitecture=MSIL">
    <SpecificVersion>False</SpecificVersion>
    <HintPath>[PATH]\HOOKS-Assembly-
CSharp.dll</HintPath>
  </Reference>
</ItemGroup>
```

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFrameworks>netstandard2.0;net350</TargetFrameworks>
  </PropertyGroup>
  <PropertyGroup Condition=" '$(TargetFramework)' == 'net350' ">
    <FrameworkPathOverride>$(Mono) \lib\mono\
  </PropertyGroup>
  <ItemGroup>
    <Reference Include="Assembly-CSharp">
      <HintPath> \Assembly-CSharp.dll</HintPath>
    </Reference>
    <Reference Include="ConfigMachine">
      <HintPath> \ConfigMachine.dll</HintPath>
    </Reference>
    <Reference Include="HOOKS-Assembly-CSharp, Version=0.0.0.0, C
      <SpecificVersion>False</SpecificVersion>
      <HintPath> \HOOKS-Assembly-CSharp.dll</HintPath>
    </Reference>
  </ItemGroup>
```

```

<Reference Include="MonoMod">
  <HintPath>[PATH]\MonoMod.exe</HintPath>
</Reference>
<Reference Include="Partiality">
  <HintPath>[PATH]\Partiality.dll</HintPath>
</Reference>
<Reference Include="UnityEngine">
  <HintPath>[PATH]\UnityEngine.dll</HintPath>
</Reference>
</ItemGroup>

```

Click Class1.cs automatically created. The rest is the same! Follow the instruction above regarding how to make partiality mods.

To build the mod, however, go to Solution Explorer and right-click your solution, and click Build. In bin\Debug\net350\ your mod will be built. If you want to edit the version which appears when you mouse over the file, however, open \obj\Debug\net350\[YourModName].AssemblyInfo.cs and edit them.

Now you are ready to build Rain World mods with VS Code!

Of course, you can avoid Visual Studio completely and use something like Atom, Vim or JOE, but I cannot explain something I've never used.

-topicular